



**INDIVIDUAL ASSIGNMENT
INTERNET OF THINGS PROJECT
CT150-3-2-IOTC**

**NEXLIVING
SMART HOME MONITORING**

LECTURER NAME: MR. KAMALANATHAN SHANMUGAM

DATE ASSIGNED: 7 DEC 2025

DATE COMPLETED: 22 FEB 2026

NAME: NUR HANISA DASIRA BINTI MOHD SUHAIMI

TP NUMBER: TP077280

INTAKE CODE: APU2511IT (IOT)

Table of Contents

1.0 Introduction.....	3
1.1 Project Background.....	3
1.2 Project Objectives	4
1.3 Key Features of the System	4
2.0 IoT Components	6
2.1 Hardware Component.....	6
2.3 Software Component.....	10
2.3.1 Arduino IDE	10
2.3.2 ThingSpeak.....	11
3.0 Source Code	12
4.0 System Output	21
4.1 Thing Speak.....	21
4.2 Web Server	25
5.0 NexLiving Prototype	27
6.0 Conclusion.....	28
7.0 References	29

1.0 Introduction

The rapid advancement of Internet technologies has led to the emergence of the Internet of Things (IoT). This technology enables physical devices to collect, transmit, and exchange data over the Internet with minimal human intervention. Increasingly, various industries are incorporating IoT in their organizations to help them operating more efficiently by providing decision-making and enhanced customer service (Alexander Gillis, 2024).

IoT ecosystem consist of four elements for it to function which are sensors or devices, connectivity, data analytics, and graphical user interface (GUI). Firstly, sensors or devices is the most important component in IoT as it is basically a physical device or object that is embedded with sensors, software, and technologies to enable them to process the data receive and execute the system. Next, connectivity is the element which enable IoT devices to communicate with one another through a network over the internet. For example, Wi-Fi connectivity is utilized to enable data transmission and support Over-The-Air (OTA) to control the devices without physical access.

Moreover, the data analytics element in IoT used to identify patterns, offer recommendations and identify potential issues. Some physical devices may handle a large volume of data, and this would cause a troublesome as it would be a huge bandwidth consumption. Hence, data can be analysed locally to reduce this issue. Lastly, in managing IoT devices without any physical access needed, GUI is built in website or mobile app to manage, control, and register smart devices. This project using all four elements of IoT for an automated response, alert and web-based monitoring interfaces.

Overall, this project demonstrates all the elements of IoT by integrating sensors, wireless connectivity, local data processing, and automated alert mechanisms. Thus, this project fulfils the objectives of real-time data acquisition, intelligent decision-making, and reliable system maintenance.

1.1 Project Background

In today's technological revolution lies the concept of the Smart Home, an innovative ecosystem where devices in the household communicate through Internet of Things (IoT) to provide convenience and security (TDK, 2024). Issues such as hazardous gas leakage, excessive temperature, and high humidity can pose as possible risks to human health and comfort. The absence of technology like IoT, traditional monitoring systems are needed to control these issues, and it is often requiring manual intervention and lack real-time responsiveness.

With the advancement of Internet of Things (IoT) technology, these smart devices are no longer just about convenience but also essential tools to enhance safety in home (Anon, 2025). The core of this technology is IoT as the foundation of smart homes which connects sensors, actuators, and devices to automate and the functions. Next, to enhance living experience, IoT devices improve comfort, efficiency, and safety. Lastly, the security advancement which enables real-time monitoring, automated alerts, and access control for residential safety (TDK, 2024). Thus, by putting together these components, IoT creates smart

homes that are adaptive to environment changes and user needs to provide optimal convenience and security.

This project is developed within the context of an IoT-based environmental monitoring and safety system that integrates sensors and wireless communication technologies. By applying IoT principles, the system aims to provide continuous monitoring, automated alerts, and wireless house control and monitoring. The application of Over-The-Air (OTA) functionality in this project enhance the system by allowing remote house control to ensure long-term reliability and scalability. This approach demonstrates the practical application of IoT concepts in daily life devices and environment for a better living.

1.2 Project Objectives

The main objective of this project is to design and implement an Internet of Things (IoT) based environmental monitoring and safety system which capable to detect hazardous conditions and responds automatically in real-time. Following are the objectives of this project:

1. **To develop an IoT-based system using an ESP32 microcontroller** that integrates sensors for real-time environmental monitoring
2. To ease daily life by increasing user comfort by enabling home control remotely. The ability to control electronics from smartphone, tablet, or laptop, adds an extra level of convenience.
3. **To monitor temperature and humidity levels using DHT22 sensor** and activates aircons when the predefined temperature and humidity are exceeded.
4. **To apply appropriate wireless sensor network and communication principles** by processing sensor data locally on the microcontroller for an efficient system response.
5. **To demonstrate the practical application of IoT concepts** including data acquisition, real-time analysis, and remote control.

1.3 Key Features of the System

The proposed IoT-based system to provide an efficient, reliable, and automated solution for environmental monitoring and safety management. The integration of sensors with a microcontroller and automated alert mechanisms, the system provides features such as detecting hazardous conditions and respond in real-time. Following are the key features of the system which highlights the core functionalities:

1. Real-time Gas Detection and Alert System

The system continuously monitors the environment for the presence of the hazardous gas such as VOC and H₂ based CO₂ using a gas sensor. When gas concentration exceeds a safe level, an audible alert from the buzzer will immediately be triggered to warn occupants and enhance safety.

2. Temperature and Humidity Monitoring

DHT22 sensor is used to measure real-time temperature and humidity levels within the environment. The system continuously monitors the environment to ensure that the house is comfortable by automatically turns on the air-conditioning if the measured temperature or humidity exceed predefined threshold values and turns off once the environment returns to normal conditions.

3. Hybrid Local and Cloud Data Processing

The processing of sensor data will be processed locally on the ESP32 to enable immediate responses such as alerts and indicators to ensure low latency and reliability. At the same time, selected sensor data is transmitted to ThingSpeak cloud platform for real-time visualization, remote monitoring, and analysis. This hybrid approach improves the scalability and usability of the system while preserving fast response times.

4. Wireless and Scalable IoT Architecture

The system is built using IoT design principles and wireless communication technologies to allow future expansion such as remote monitoring and controlling and cloud integration.

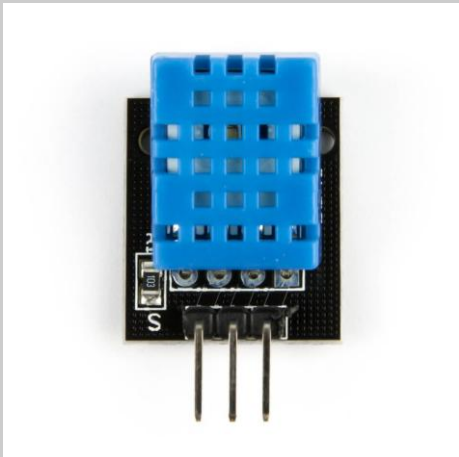
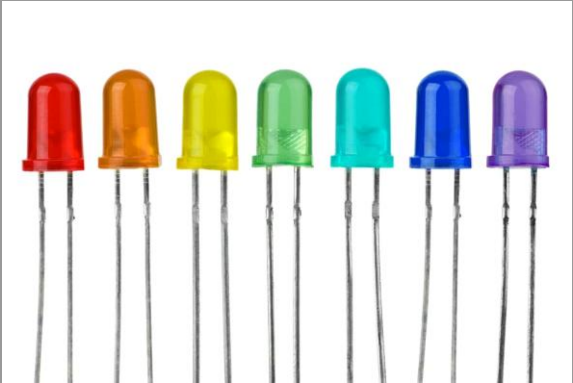
2.0 IoT Components

In enabling physical devices to collect, process, and exchange data over the internet with minimal human intervention, IoT system is composed with multiple interconnected components which plays crucial roles in ensuring reliable data acquisition, processing, and outcome. It can be divided into two categories which are hardware and software. Hardware in IoT performs multiple functions in the system such as system launch and security, action specifications, communication, and detection of support-specific goals and actions (Anon, IoT Hardware and its place in the IoT ecosystem, 2021). Meanwhile, the software components is a platform, firmware, or operating systems that function to manage, analyse, and enable communication in the system. Hence, the integration of hardware and software in IoT helps in gathering, processing, transmit data, and act out the commands from the system for the goal of the project to be achieved.

2.1 Hardware Component

The hardware components of an Internet of Things (IoT) system are the physical foundation that enables the interaction with the environment. These components are responsible for collecting and transmitting data that can be used to derive insights and improve decision-making (Anon, IoT Hardware and its place in the IoT ecosystem, 2021). Hardware in IoT includes sensors, actuators, microcontrollers, and communication interfaces. These components work together to ensure accurate data collection and reliable system operation. The following are the components used in the NexLiving project:

No.	Hardware Component	Function
1.	ESP32  <i>Figure 2.1.1 ESP32 (Anon, 2023)</i>	ESP32 is a chip that provides Wi-Fi and Bluetooth connectivity for embedded devices. - Act as the main microcontroller - Collects data from connected sensors which are DHT11 and SGP30 - Controls actuators components such as LEDs, buzzers, relays, and servo motors - Enables wireless connection using built-in Wi-Fi and Bluetooth to carry out Over-The-Air (OTA) and cloud platform's function

2.	DHT11  <i>Figure 2.1.2 DHT 11 (Anon, 2026)</i>	- To measure relative humidity and temperature - In this project, DHT11 will automatically turn on air-conditioning, if the temperature and humidity exceeded the maximum threshold and turn off when it goes back to normal
3.	LED  <i>Figure 2.1.3 LED (Wan, 2024)</i>	LED stands for light emitting diode which is a lighting product that light up to 90% more efficiently than incandescent light bulbs (Star, 2025). - LED act as the light bulb of the house that can be remotely control from the web server - It is also indicated as Airconditioning. If the temperature and humidity of the house exceed the threshold, it will automatically turn on. Moreover, the air-conditioning also can be control remotely.
4.	Grove – VOC and eCO2 Gas Sensor (SGP30)	The SGP30 is a metal oxide gas sensor which has a multiple sensing elements on a single core. It has four gas sensing elements and has a fully calibrated air quality output signal which is primarily for air quality detection (Anon, Grove VOC and eCO2 Gase Sensor (SGP30) Seed

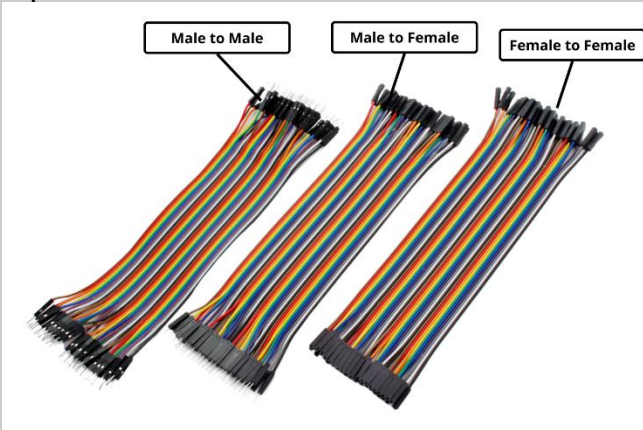
	 <i>Figure 2.1.4 SGP30 (Anon, 2026)</i>	Studio Wiki, 2023). Hence, the lower the oxygen concentration, - Air-quality monitoring as it detects presence of VOCs and provides CO2 equivalent signals
5.	Piezo Buzzer  <i>Figure 2.1.5 Buzzer (Anon, 2019)</i>	Piezo buzzer is an electronic device that produce a tone, alarm, or sound. - It functions as an alarm if the air-quality exceeded the VOC and CO2 fixed threshold
6.	Jumper Wires  <i>Figure 2.1.6 Jumper Wires (Anon, 2026)</i>	Jumper wires is a metal conductors that has connectors pins at each end to connect two points in a circuit (Wiltronics, 2022). It comes with various colours to help you track of what is connected to which. For example, red indicates power and black indicated ground.
7.	Servo Motor	Servos are controlled by sending an electrical pulse width modulation (PWM), through the control wire. A servo motor usually can only turn 90 degrees in either direction or a total of 180 degrees (Anon, 2026). - It is function to open and closing the door



Figure 2.1.7 Servo Motor (Emmanuel, 2020)

by rotating 90 degrees

8. Breadboard

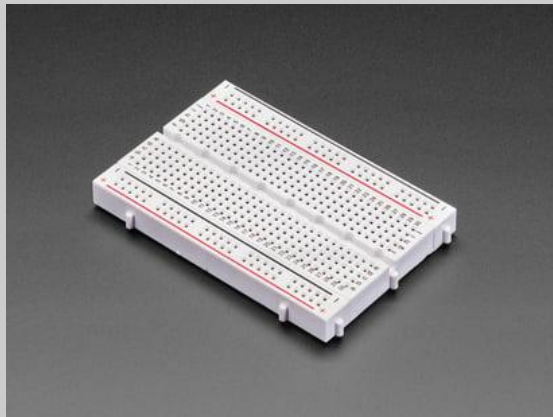


Figure 2.1.8 Breadboard (Anon, Untitled, 2026)

Breadboard is the foundation to construct and prototype electronics (Anon, n.d.). It enables developers to easily connect components or wires following the rows and columns of the board.

9. Arduino UNO

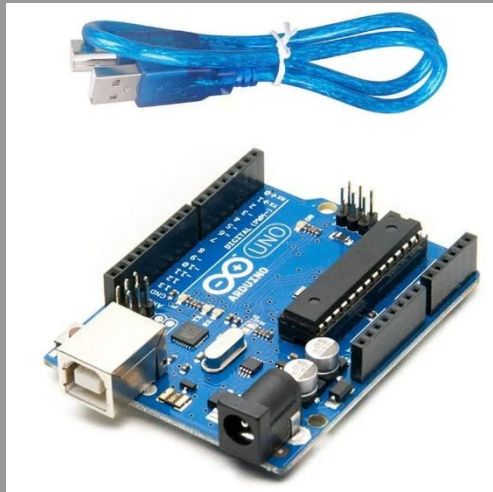


Figure 2.1.9 Arduino UNO (Anon, 2025)

Arduino UNO is a microcontroller board based on the ATmega328)

- Function as extra power supply for servo motor

2.3 Software Component

The software component of NexLiving is important for communication, data processing, device control, and user interaction. In this project, the software components used are Arduino IDE and ThingSpeak. These components are integrated to enable monitoring, control, and remote management of connected devices.

2.3.1 Arduino IDE



Figure 2.3.1.1 Arduino IDE

Arduino IDE is a primary software platform for developing, compiling, and uploading programs to the ESP32 microcontroller for this project. It is an open-source software that provides simple and user-friendly interface that supports both C and C++ language. It also enables the integration of multiple libraries for sensor interfacing, wireless communication, and cloud connectivity (Anon, n.d.).

NexLiving applies Over-The-Air (OTA) updates which allows software, firmware, or configuration update to be delivered wirelessly with minimal physical intervention. This ensures continuous improvements without disrupting the device's functionality (Kashyap, 2024). The application of OTA suitable for smart homes, NexLiving which is a system that is installed in a private or hard-to-reach location. Hence, OTA reduces maintenance time, downtime, and improves system reliability by allowing updates without physical intervention.

Moreover, Arduino IDE also enable web server application such as the ESP32 to host a web page that users can access through a standard web browser (Santos, 2025). This embedded web server allows user to remotely control devices and monitor sensor data in real time. For example, in NexLiving, user able to control door, light, and air-conditioning remotely and monitor the house temperature and air quality. These commands can be controlled through HTTPS requests, and this provides a simple and cost-effective user interface for remote monitoring and control.

In conclusion, the integration of OTA and web server application in Arduino IDE provides an efficient software solution for this project which allows enhancement in the system functionality and scalability to improve NexLiving following user's requirement.

2.3.2 ThingSpeak

ThingSpeak is a communication library for Arduino, ESP32, and ESP8266. It is an analytic IoT platform service that allow user to aggregate, visualize, and analyse live data streams in the cloud (Anon, 2026). In this project, ThingSpeak functions as a cloud-based data logging and monitoring platform. The sensor data from ESP32 will be send through WIFI using HTTP requests and stores the data in predefined fields within a ThingSpeak channel.

The channel will visualize the information in the form of real-time graphs and charts. From this, user can remotely monitor environmental conditions, The stored data can be analysed to identify trends, detect abnormal conditions, and monitor system performance. As an example, ESP32 reads sensor data such as temperature, humidity, TVOC, and eCO₂. It also reads the state of lights and doors to be transmitted to ThingSpeak. All in all, ThingSpeak enable features that are informative and reliable for NexLiving.

3.0 Source Code

```
1  #include <Arduino.h>
2  #include <WiFi.h>
3  #include <HTTPClient.h>
4  #include <ArduinoOTA.h>
5  #include <DHT.h>
6  #include <ESP32Servo.h>
7  #include <Wire.h>
8  #include <Adafruit_SGP30.h>
```

Figure 3.1 Libraries for ESP32

```
10 // wifi
11 const char* ssid = "nasi (2)";
12 const char* password = "hanisadasira";
13 WiFiServer server(80);
14
```

Figure 3.2 Wi-Fi Setup

```
15 // thingspeak
16 String apiKey = "IHWYE4DDPQXQ8EPK";
17 String channelURL = "http://api.thingspeak.com/update";
18 unsigned long lastTS = 0;
19 bool thingSpeakOK = false;
```

Figure 3.3 ThingSpeak Setup

```
21 // define pins
22 #define DHT_PIN      4
23 #define LED_PIN      5
24 #define AIRCOND_PIN  26
25 #define BUZZER_PIN   27
26 #define SERVO_PIN    25
```

Figure 3.4 Define Pins

```
27
28 DHT dht(DHT_PIN, DHT11);
29 Servo doorServo;
30 Adafruit_SGP30 sgp;
31
```

Figure 3.5 Sensor and Object Initialization

```
32 float temp = NAN;
33 float hum = NAN;
34 uint16_t tvoc = 0;
35 uint16_t eco2 = 0;
36
37 unsigned long lastDHTRead = 0;
38 unsigned long lastSGPRead = 0;
```

Figure 3.6 Declare Sensor Variable

```
40 // define states
41 int lightState = 0;
42 int aircondState = 0;
43 int doorState = 0;
44
45 // temp and humid threshold
46 float TEMP_LIMIT = 30.0;
47 float HUM_LIMIT = 70.0;
48
49 void setup() {
50     Serial.begin(115200);
51
52     pinMode(LED_PIN, OUTPUT);
53     pinMode(AIRCOND_PIN, OUTPUT);
54     pinMode(BUZZER_PIN, OUTPUT);
55
56     digitalWrite(LED_PIN, LOW);
57     digitalWrite(AIRCOND_PIN, LOW);
58
59     dht.begin();
60
61     doorServo.attach(SERVO_PIN);
62     doorServo.write(0);
```

Figure 3.7 States and Threshold

```
63
64   Wire.begin(21, 22);
65   delay(1000);
66
67   // check if sgp30 functioning or not
68   if (!sgp.begin()) {
69     Serial.println("SGP30 NOT detected");
70   } else {
71     Serial.println("SGP30 detected");
72   }
73
74   // connecting to wifi
75   WiFi.begin(ssid, password);
76   Serial.print("Connecting to WiFi...");
77   while (WiFi.status() != WL_CONNECTED) {
78     delay(500);
79     Serial.print(".");
80   }
81
82   //web server ip address
83   Serial.println("\nWiFi Connected");
84   Serial.print("IP Address: ");
85   Serial.println(WiFi.localIP());
86
87   server.begin();
88
89   ArduinoOTA.setHostname("ESP32-SmartHome");
90   ArduinoOTA.begin();
91
92   Serial.println("System Ready");
93 }
94
```

Figure 3.8 Sensor and Wi-Fi Preparation

```
95   void loop() {
96     ArduinoOTA.handle();
97     readDHT();
98     readSGP();
99     handleWeb();
100    climateAlert();
101    thingSpeakUpdate();
102  }
```

Figure 3.9 loop() Function

```
104 // dht 11 sensor
105 void readDHT() {
106     if (millis() - lastDHTRead < 2000) return;
107
108     float t = dht.readTemperature();
109     float h = dht.readHumidity();
110
111     if (!isnan(t) && !isnan(h)) {
112         temp = t;
113         hum = h;
114     }
115
116     lastDHTRead = millis();
117 }
```

Figure 3.10 readDHT()

```
119 // check air quality
120 void readSGP() {
121     if (millis() - lastSGPRead < 1000) return;
122
123     if (sgp.IAQmeasure()) {
124         tvoc = sgp.TVOC;
125         eco2 = sgp.eCO2;
126     }
127
128     lastSGPRead = millis();
129 }
130
131 // sgp30 alert
132 void climateAlert() {
133     bool alert = false;
134
135     if (!isnan(temp) && temp > TEMP_LIMIT) alert = true;
136     if (!isnan(hum) && hum > HUM_LIMIT) alert = true;
137     if (eco2 > 1000) alert = true;
138
139     digitalWrite(BUZZER_PIN, alert);
140 }
141
```

Figure 3.11 readSGP() and sgp30() Alert

```
142 // web server
143 void handleWeb() {
144     WiFiClient client = server.available();
145     if (!client) return;
146
147     String header = "";
148
149     while (client.connected()) {
150         if (client.available()) {
151             char c = client.read();
152             header += c;
153
154             if (c == '\n') {
155
156                 // button handling
157                 if (header.indexOf("GET /light/on") >= 0) {
158                     digitalWrite(LED_PIN, HIGH); lightState = 1;
159                 }
160                 if (header.indexOf("GET /light/off") >= 0) {
161                     digitalWrite(LED_PIN, LOW); lightState = 0;
162                 }
163                 if (header.indexOf("GET /aircond/on") >= 0) {
164                     digitalWrite(AIRCOND_PIN, HIGH); aircondState = 1;
165                 }
166                 if (header.indexOf("GET /aircond/off") >= 0) {
167                     digitalWrite(AIRCOND_PIN, LOW); aircondState = 0;
168                 }
169                 if (header.indexOf("GET /door/open") >= 0) {
170                     doorServo.write(0); doorState = 1;
171                 }
172                 if (header.indexOf("GET /door/close") >= 0) {
173                     doorServo.write(180); doorState = 0;
174                 }
175             }
176         }
177     }
178 }
```

Figure 3.12 Web Server


```
174     }
175
176
177     client.println("HTTP/1.1 200 OK");
178     client.println("Content-type:text/html");
179     client.println("Connection: close\n");
180
181
182     client.println(R"rawliteral(
183 <!DOCTYPE html>
184 <html>
185 <head>
186 <meta name="viewport" content="width=device-width, initial-scale=1">
187 <meta http-equiv="refresh" content="5">
188 <title>Smart Home</title>
189
190 <style>
191 body {
192     font-family: 'Segoe UI', Arial, sans-serif;
193     background: #c7d5f0;
194     margin: 0;
195 }
196 .container {
197     max-width: 420px;
198     margin: auto;
199     padding: 20px;
200 }
```

Figure 3.13 Web Server

```
201 h1 { text-align: center; }
202 .card {
203     background: #FFB6C1;
204     border-radius: 14px;
205     padding: 15px;
206     margin-bottom: 15px;
207     box-shadow: 0 4px 10px rgba(0,0,0,0.08);
208 }
209 .status { text-align: center; font-weight: bold; }
210 .ok { color: #43a047; }
211 .error { color: #e53935; }
212 .wait { color: #fb8c00; }
213 .buttons {
214     display: grid;
215     grid-template-columns: 1fr 1fr;
216     gap: 10px;
217 }
218 .btn {
219     padding: 14px;
220     border-radius: 10px;
221     color: white;
222     text-decoration: none;
223     font-weight: bold;
224     text-align: center;
225 }
226 .on { background: #4CAF50; }
227 .off { background: #F44336; }
228 </style>
229 </head>
230
```

Figure 3.14 Web Server

```

231 <body>
232 <div class="container">
233 <h1>NexLiving</h1>
234
235 <div class="card status">
236 )rawliteral";
237
238     // check if sensor is working
239     if (isnan(temp) || isnan(hum)) {
240         client.println("<p class='error'>DHT SENSOR ERROR</p>");
241     } else {
242         client.println("<p class='ok'>Temperature & Humidity OK</p>");
243     }
244
245     client.println("</div><div class='card'>");
246
247     // display sensor data
248     if (!isnan(temp) && !isnan(hum)) {
249         client.print("<p>Temperature: ");
250         client.print(temp);
251         client.println(" &deg;C</p>");
252
253         client.print("<p>Humidity: ");
254         client.print(hum);
255         client.println(" %</p>");
256     }
257

```

Figure 3.15 Web Server

```

258     client.print("<p>TVOC: ");
259     client.print(tvoc);
260     client.println(" ppb</p>");
261
262     client.print("<p>eCO&#8322;: ");
263     client.print(eco2);
264     client.println(" ppm</p>");
265
266     client.println("</div><div class='card status'>");
267
268     // ===== THINGSPEAK STATUS =====
269     client.print("ThingSpeak: ");
270     if (thingSpeakOK) {
271         client.println("<span class='ok'>UPDATED</span>");
272     } else {
273         client.println("<span class='wait'>WAITING</span>");
274     }
275
276     client.println("</div><div class='card'>");
277
278     // states
279     client.println("<p>Light: " + String(lightState ? "ON" : "OFF") + "</p>");
280     client.println("<p>Aircond: " + String(aircondState ? "ON" : "OFF") + "</p>");
281     client.println("<p>Door: " + String(doorState ? "OPEN" : "CLOSED") + "</p>");
282
283     client.println("</div><div class='card buttons'>");

```

Figure 3.16 Web Server

```

285 // button
286 client.println("<a class='btn on' href='/light/on'>Light ON</a>");
287 client.println("<a class='btn off' href='/light/off'>Light OFF</a>");
288 client.println("<a class='btn on' href='/aircond/on'>Aircond ON</a>");
289 client.println("<a class='btn off' href='/aircond/off'>Aircond OFF</a>");
290 client.println("<a class='btn on' href='/door/open'>Door OPEN</a>");
291 client.println("<a class='btn off' href='/door/close'>Door CLOSE</a>");
292
293 client.println("</div></div></body></html>");
294 break;
295 }
296 }
297 }
298 client.stop();
299 }
300

```

Figure 3.17 Web Server

```

301 // thingspeak
302 void thingSpeakUpdate() {
303     if (millis() - lastTS < 20000) return;
304     if (WiFi.status() != WL_CONNECTED) return;
305
306     HTTPClient http;
307     String url = channelURL +
308         "?api_key=" + apiKey +
309         "&field1=" + String(temp) +
310         "&field2=" + String(hum) +
311         "&field3=" + String(tvoc) +
312         "&field4=" + String(eco2) +
313         "&field5=" + String(lightState) +
314         "&field6=" + String(doorState);
315
316     http.begin(url);
317     int httpCode = http.GET();
318     http.end();
319
320     thingSpeakOK = (httpCode == 200);
321     lastTS = millis();
322 }

```

Figure 3.18 ThingSpeak

4.0 System Output

The system output in NexLiving is the responses or actions produced by the Smart Home System after processing input data from sensors and user commands. In this ESP32-based project, outputs are generated based on real-time environmental readings and safety conditions received through the web interface.

The system produces both physical outputs and digital outputs. The physical outputs are the activation of the LED light, air-conditioner, servo motor for door control, and buzzer alert. These components respond are some automatics or can be manually controlled by the user via the web server. In addition, the digital outputs such as live sensor reading displayed on the web server and will be updated to the cloud platform, ThingSpeak. This allows users to monitor environmental condition remotely and the data trends. Following are the display for ThingSpeak and web server for NexLiving.

4.1 Thing Speak

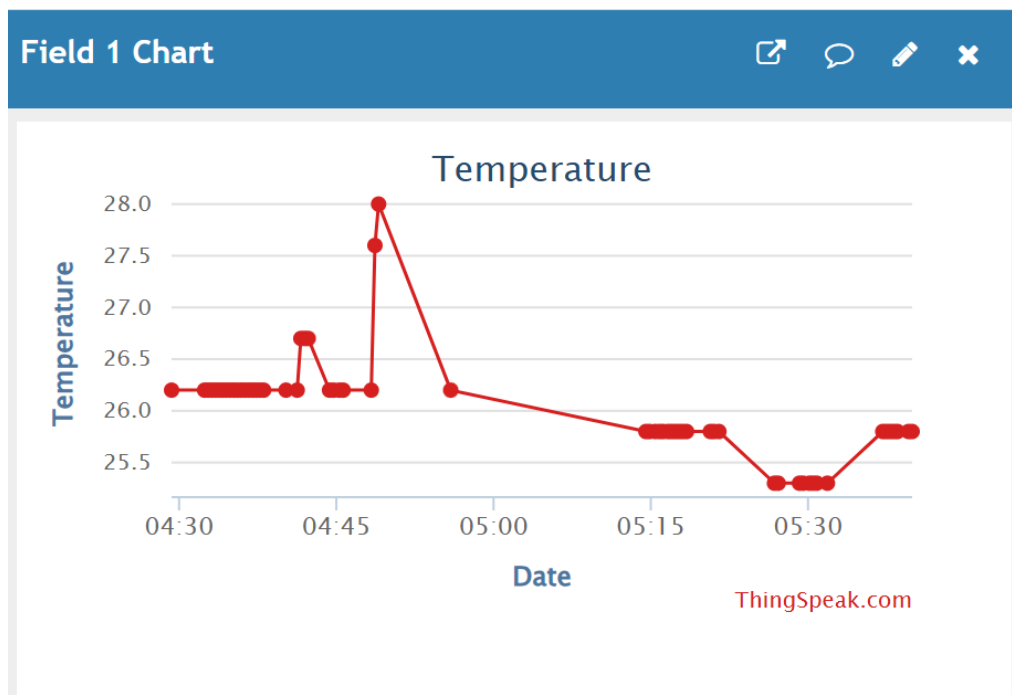


Figure 4.1.1 ThingSpeak for Temperature Chart

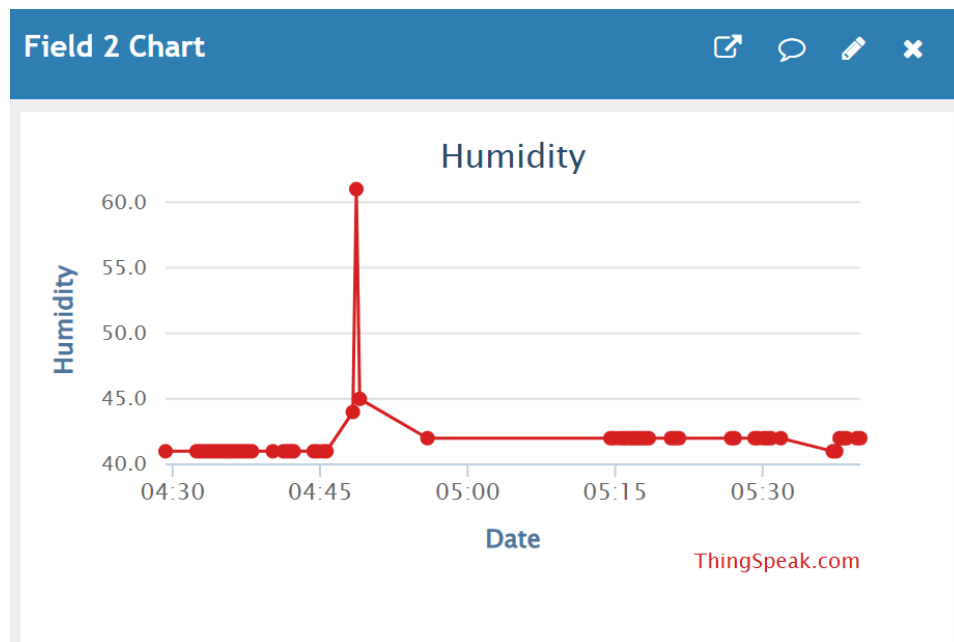


Figure 4.1.2 ThingSpeak for Humidity Chart

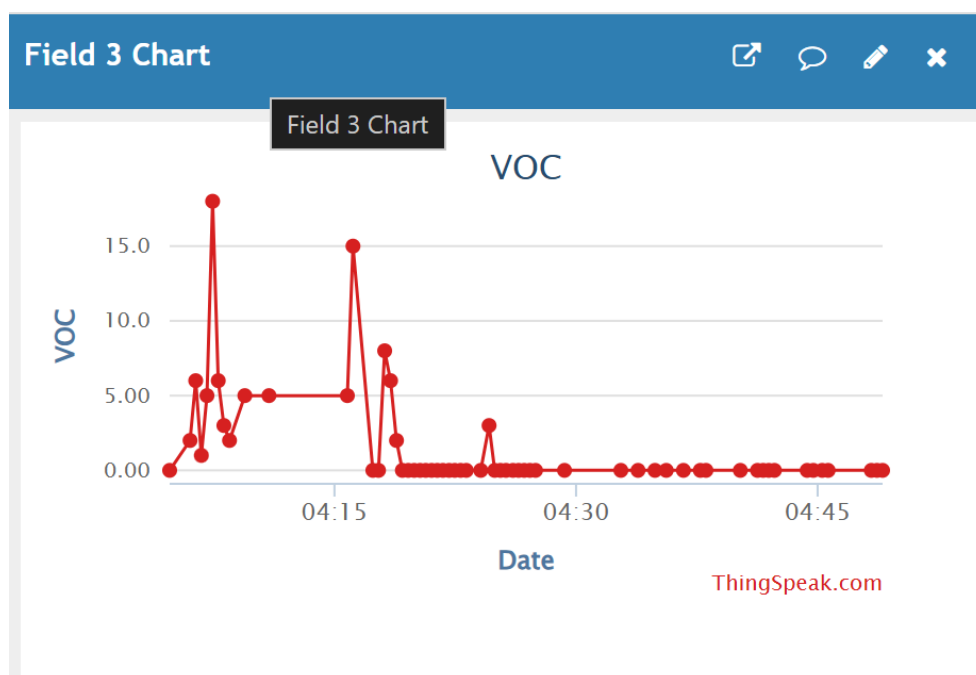


Figure 4.1.3 ThingSpeak for VOC Chart

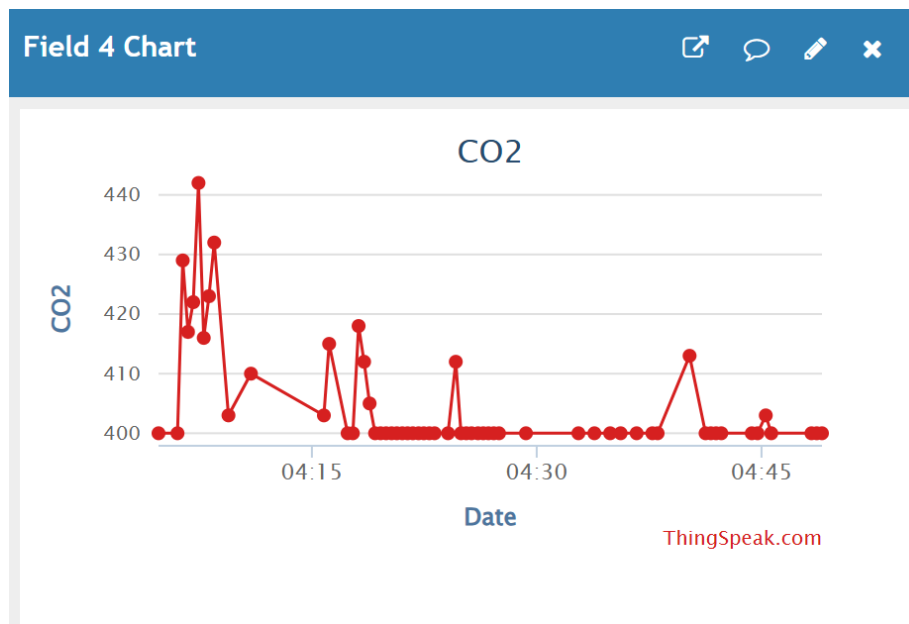


Figure 4.1.4 ThingSpeak for CO₂ Chart

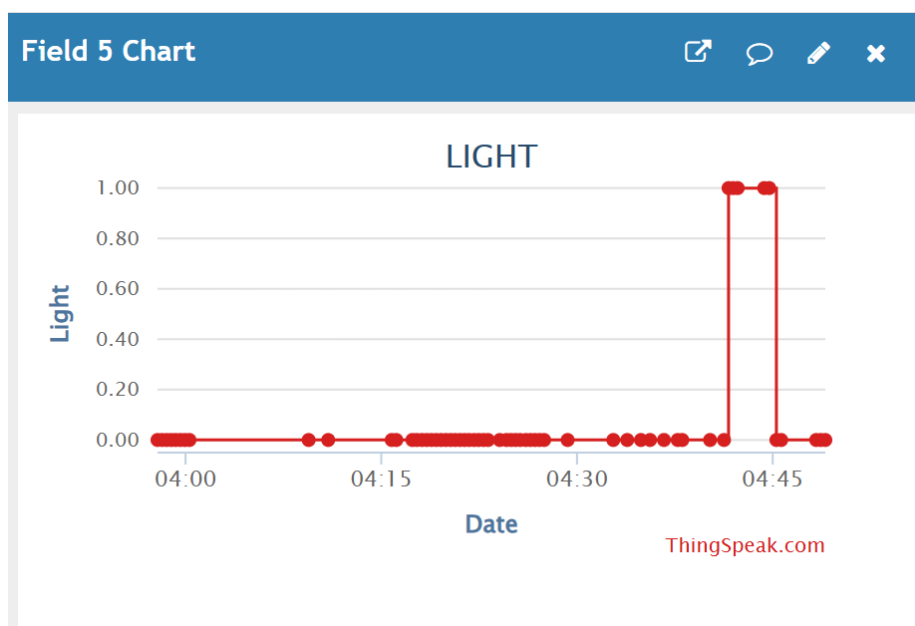


Figure 4.1.5 ThingSpeak for Light Chart

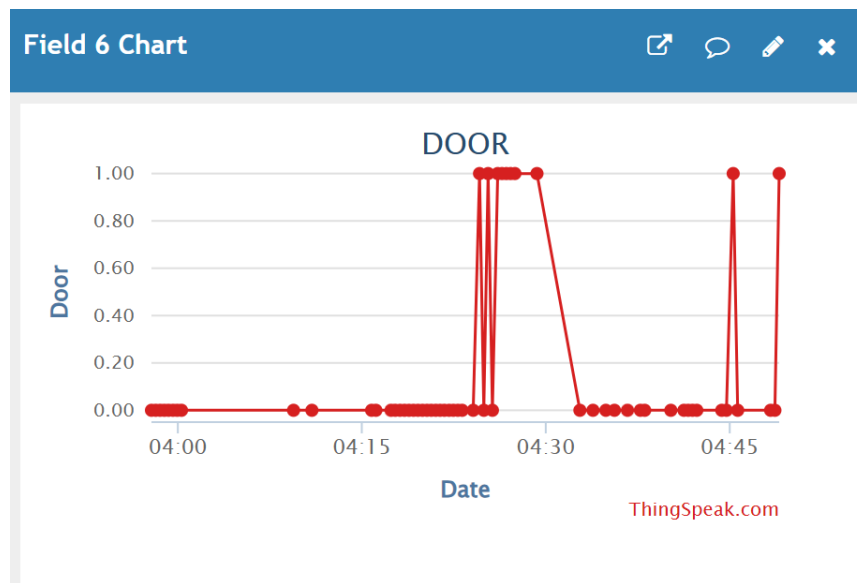


Figure 4.1.6 ThingSpeak for Door Chart

4.2 Web Server

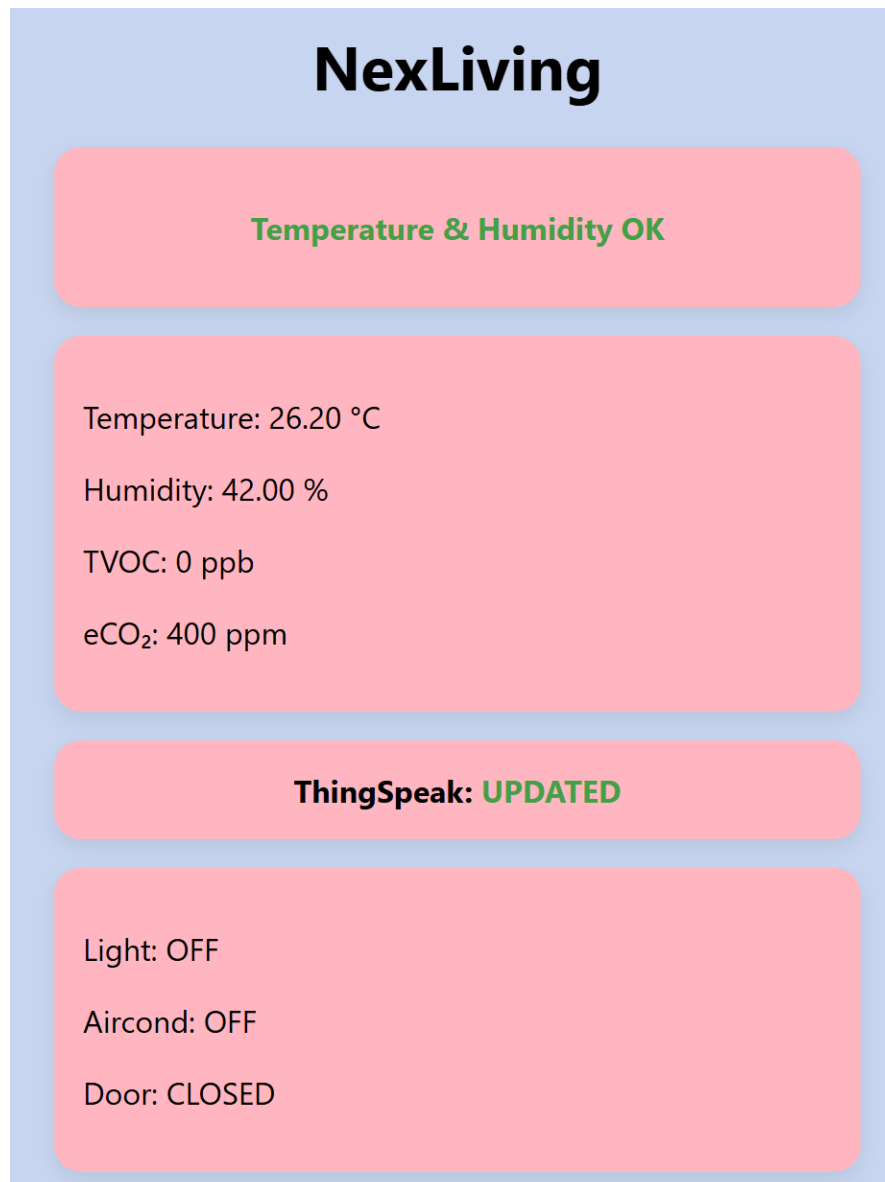


Figure 4.2.1 Web Server

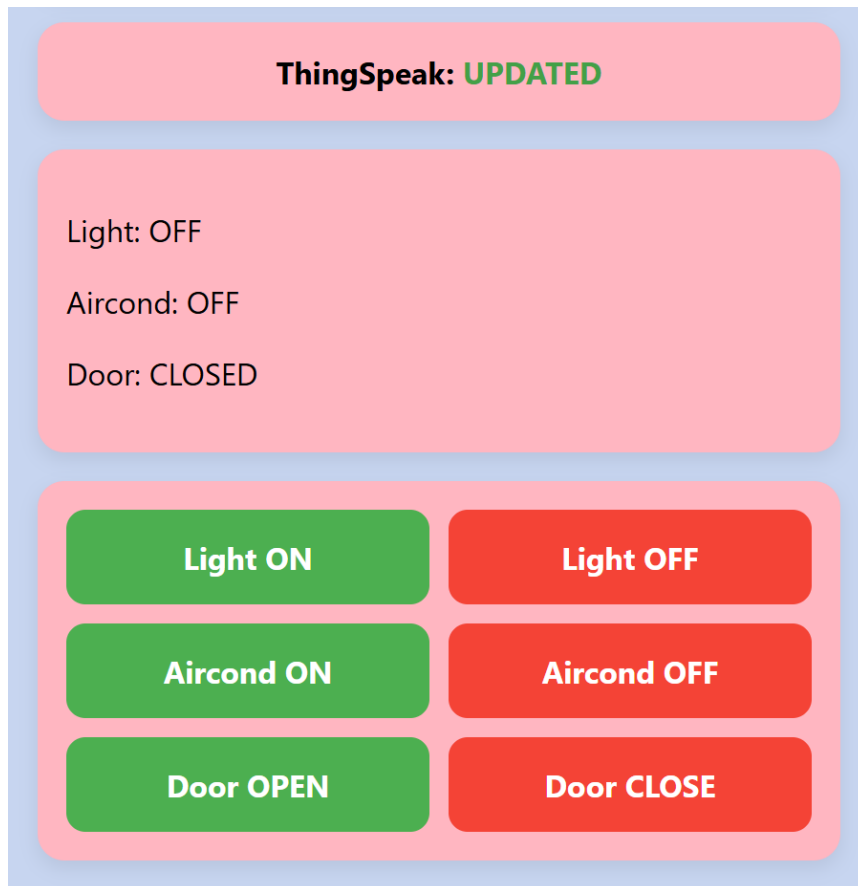


Figure 4.2.2 Web Server

5.0 NexLiving Prototype

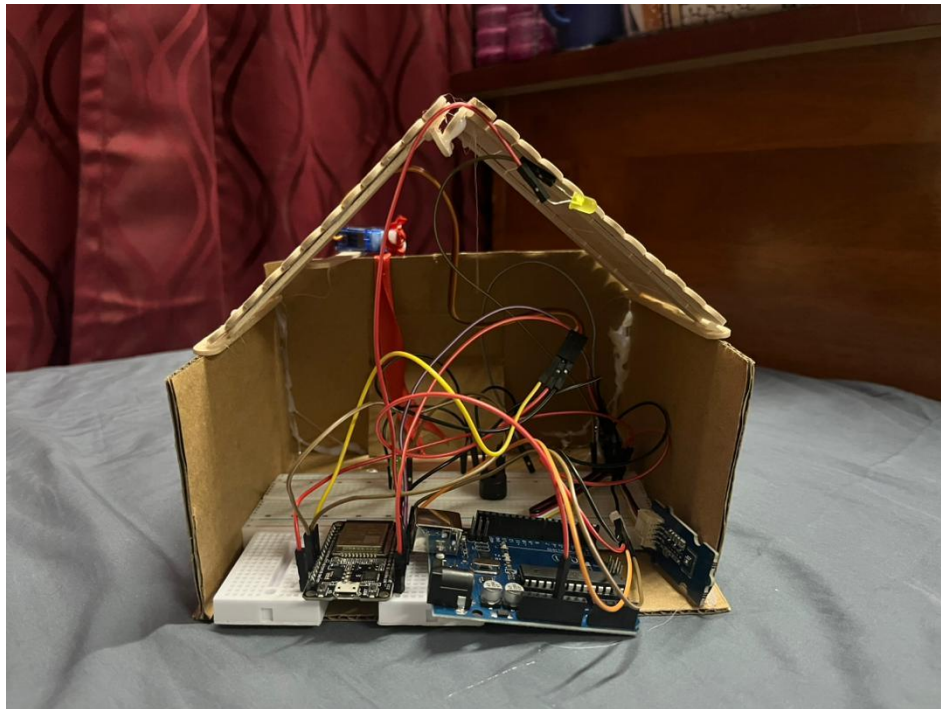


Figure 5.1 Back Part



Figure 5.2 Front Part

6.0 Conclusion

In conclusion, NexLiving, ESP-32 based Smart Home Monitoring and Control System integrates environmental sensing, device automation, web-based control, and cloud connectivity into a platform. The system continuously monitors temperature, humidity, TVOC, and eCO2 levels using DHT11 and SGP30 sensors.

The implementation of smart house monitor enhances system reliability by having automatic sensor for convenience and safety purposes. Based on this project, the sensors used are DHT11 and SGP30 where DHT11 monitor the house temperature and humidity and SGP30 monitor the air-quality. If SGP30 detected low air quality in the house environment, it will produce an alert from the buzzer. This feature improves user safety by providing immediate warnings in case of any potentially dangerous conditions.

In addition, the web-based interface displays the real-time reading of the house temperature, humidity, and reading of the SGP30. This allows user to monitor the house environment, and the household devices can be remotely controlling such as lighting, air-conditioning, and door access through a Wi-Fi connection. The integration with ThingSpeak further extends the system by enabling cloud data logging and remote monitoring from anywhere.

Overall, NexLiving shows an effective combination of IoT technology, automation, and real-time monitoring by providing a practical, scalable, and user-friendly smart home solution that enhances a convenience and safety of the house.

7.0 References

- Alexander Gillis, K. Y. (2024). *What Is IoT (Internet of Things) and How Does It Work?* Retrieved from TechTarget: <https://www.techtarget.com/iotagenda/definition/Internet-of-Things-IoT>
- Anon. (2019). *NodeMCU Peizo Buzzer on Arduino IDE*. Retrieved from Sriparna's blog on Internet of Things: <https://sriparnaiot.wordpress.com/nodemcu-peizo-buzzer-on-arduino-ide/>
- Anon. (2021). *IoT Hardware and its place in the IoT ecosystem*. Retrieved from 1nce: <https://www.1nce.com/en-ap/resources/news/blog/iot-hardware-and-ecosystem>
- Anon. (2023). *ESP32 NodeMCU Module WLAN WiFi Development Board | Dev Kit C V2 With CP102 compatible with Arduino*. Retrieved from AZ-Delivery: https://www.az-delivery.de/en/products/esp32-developmentboard?srsltid=AfmBOorHc_yRr1zsHeCy5I7kuSUCc3anw1f-2tLKvY2rn_6i4JrarAbS
- Anon. (2023). *Grove VOC and eCO2 Gase Sensor (SGP30) | Seed Studio Wiki*. Retrieved from Seedstudio: <https://wiki.seeedstudio.com/K1100-VOC-and-eCO2-Gas-Sensor-Grove-LoRa-E5/>
- Anon. (2025). *Decoding the Arduino Uno Pins Layout: A Comprehensive Guide - AnyPCBA*. Retrieved from AnyPCBA: <https://www.anypcba.com/blogs/pcb-layout-knowledge/decoding-the-arduino-uno-pins-layout-a-comprehensive-guide.html>
- Anon. (2025). *From Convenience to Necessity: How IoT is Redefining Home Safety | IoT For All*. Retrieved from IoT For All: <https://www.iotforall.com/iot-home-safety>
- Anon. (2026). *DHT11 Temperature and Humidity Sensor Module Breakout*. Retrieved from Cytron Technologies: https://www.cytron.io/p-dht11-sensor-module-breakout?srsltid=AfmBOoqwvgv5N6mri6iETsgdH5ftiMJt_T9s0Qz_lMvpPraRDy47hL6
- Anon. (2026). *Grove - VOC and eCO2 SGP30 Gas Sensor*. Retrieved from Cytron Technologies Malaysia: <https://my.cytron.io/p-grove-voc-and-eco2-sgp30-gas-sensor>
- Anon. (2026). *Jumper Cable - ATL Equipment Manual*. Retrieved from AIm.gov.in: <https://atl.aim.gov.in/ATL-Equipment-Manual/jumper-cable/>
- Anon. (2026). *PlatformIO Registry*. Retrieved from PlatformIO Registry: <https://registry.platformio.org/libraries/mathworks/ThingSpeak>
- Anon. (2026). *Untitled*. Retrieved from Rizaltrophy.com.my: <https://rizaltrophy.com.my/1207252/Sized-Premium-Breadboard-830-Tie-Points-ID>
- Anon. (2026). *What is a Servo Motor and How it Works | Jameco*. Retrieved from Jameco.com: <https://www.jameco.com/Jameco/workshop/Howitworks/how-servo-motors->

work.html?srsId=AfmBOooPq_pijzDUlpMcMGyZN7hZMJY2pqKGxCUIaXetL5bzLNSn4Yh

Anon. (n.d.). *Arduino Integrated Development Environment (IDE)*. Retrieved from oit: <https://www.oit.va.gov/Services/TRM/ToolPage.aspx?tid=13681>

Anon. (n.d.). *Breadboard - an overview | ScienceDirect Topics*. Retrieved from sciencedirect: <https://www.sciencedirect.com/topics/engineering/breadboard>

Emmanuel. (2020). *Servo*. Retrieved from Spiridakis: <https://spiridakis.eu/2020/09/10/arduino-servo/>

Kashyap, A. (2024). *OTA Updates for IoT Devices: Why They Matter Today*. Retrieved from Hashstudioz.com: <https://www.hashstudioz.com/blog/the-importance-of-ota-over-the-air-updates-for-iot-devices-in-todays-world/>

Santos, S. (2025). *Building an ESP32 Web Server: The Complete Guide for Beginners | Random Nerd Tutorials*. Retrieved from Random Nerd Tutorials: <https://randomnerdtutorials.com/esp32-web-server-beginners-guide/>

Star, E. (2025). *Learn About LED Lighting*. Retrieved from ENERGY STAR: <https://www.energystar.gov/products/learn-about-led-lighting#:~:text=LED%20stands%20for%20light%20emitting,the%20result%20is%20visible%20light.>

TDK. (2024). *Smart Homes Revolution: How IoT is Making Homes Smarter and Safer*. Retrieved from Learn about Technology with TDK: <https://www.tdk.com/en/tech-mag/past-present-future-tech/how-iot-is-making-homes-smarter-and-safer>

Wan, M. (2024). *What Is The Polarity of LED Lights*. Retrieved from Ledylighting: <https://www.ledylighting.com/what-is-the-polarity-of-led-lights/>

Wiltronics. (2022). *What are Jumper Wires: Know by Colour, Types, and Uses*. Retrieved from Wiltronics: <https://www.wiltronics.com.au/wiltronics-knowledge-base/what-are-jumper-wires/>